

# Code Breakdown

## go.mod

This is the file responsible for coalescing the whole system.

It pulls the protobuf and grpc, as well as the necessary requirements.

## go.sum

This is a collection of links that are referred to in the file.

## server\_basics.go

- It defines the constants and necessary structures for the server to function. Important structs that it brings up are
  - Request Entry
  - File Key
  - Heartbeat
- The important functions that it makes are
  - `saveVersion`
    - Where it saves the version
  - `loadVersion`
    - Where it loads the default version
  - `rebuildVersionTable`
    - The function scans the directory and reconstructs the files that exists, the latest saved version and their synchronisation tools
    - It works in the following way
      - It reads all the files in the directory
      - It checks the filepath
      - If there is a `tmp` file, they will be deleted, since it indicates an incomplete write
        - This is how atomicity is "implemented", if a write is halted, it will be removed later on
      - `meta` files are used to load the latest version
      - Entry and lock conditions are reset
      - A table, which contains the metadata, is updated
      - Log recovery is updated
  - The functions `CanRead`, `CanWrite` are used to check if a file is available for usage
  - Functions such as `AcquireRead`, `ReleaseRead` are used in tandem, with each other to create tamper proof access to the files
    - Similar functions exist for writing into a file

- To ensure the Memory doesn't bloat up, an automatic and periodic garbage collector is run every 5 minutes, which cleans stale entries from cache, using the function `cleanupRequests`
- To ensure locks and other Failure Handling techs are properly working, the function `cleanupLeases` is used to find dead clients, remove them and release any file locks that they had
  - When this function works, it locks the global server state, so that it can iterate over the whole server
  - It checks the metadata of each locked file, which allows it to check the client's lease. Using a heartbeat check, we can see if the lease is currently in use or if the lease is expired
  - After storing the list of expired leases and those clients, these clients are removed from the `openMap`
- `getFileEntry` is used to safely access the file
  - It locks the shared map to avoid race conditions
  - If the file entry doesn't exist, it's created
  - The system ensures that `cond` is always valid, as if it's nil, it will simply update it
- `create` function is used to create files in a seamless manner. It is a leader-only, concurrency safe, idempotent file creation handler.
  - It returns a file descriptor
  - Now, it checks if the leader is calling the function. If it's not the leader, then the function doesn't work.
  - It checks the cache, checking if the request has already been made, which prevents duplicate file creation
  - The path is sanitised for the following reasons
    - It restricts generated files only to the output directory
    - It normalises the path so that the format is consistent with other files
  - The lock is granted to the file creator, allowing them to modify the file
  - Creation is atomic since it checks if all the flags are set properly, such as `os.O_CREATE` , `os.O_EXCL` and `os.O_RDWR`
  - Then, the global structure of the server is updated
    - The server is locked
    - The number of files is checked
    - The file descriptor is checked
    - The version is initialised at 0
    - The metadata is updated, updating the server maps
    - Finally, the client lease is created
- `delete` is a safe, idempotent delete file function for our RPC handler
  - It functions similarly to the `create` function, where it checks if the leader is initiating the delete
  - It ensures that input files cannot be deleted
  - It checks if the file exists to ensure idempotent delete behaviour

- It also checks if the file is currently in use. If it is being actively read or written, the file is not deleted
- The metadata is removed, removing stale metadata
- **Open** is a safe and idempotent way to open files
  - Only the leader can open the files
  - Cache is requested to ensure that if the same request comes again, duplicate opens can be opened
  - Path is normalised
  - Path entry is accessed
  - Input files can only be read
  - A file can have multiple readers, no blocking there
  - File data is tracked and stored in the map
- **Close** works like this: It checks if the user modified the file, if so, it will commit the write safely across replicas, update the file, increment the version and then clean up the client state
  - Only the leader can commit writes
  - Cache is requested to ensure idempotency
  - Write permission is accessed
  - A replication log is created, which is persisted to the disk, so ensure durability
  - This is replicated to the followers, which is appended if there is majority
  - The data is parsed and then written to the temp file, which is then copied to the actual file. After the copy, the temp file is deleted
  - Version number is updated and the commit is updated to the state machine
- **Read** function is used to view a file chunk by chunk if the client has a valid licence
  - The function is idempotent
  - It builds a unique handler
  - It gets the file's metadata and then verifies the client's registration
  - The file's metadata is locked to protect the file
  - The file pointer is reset as a way to normalise new function instances
  - The file is divided into chunks, which is fed to the client via a loop
  - Lock is held during the full read
- **Write** function is used to write the files
  - Only the leader can perform writes
  - Idempotency is maintained using cache requests
  - Using the file descriptor, the file is looked up
  - The client is validated and then the state is extracted (the locks and version)
  - The function ensures the write lock is acquired
  - A log entry is created, which is persisted to the disk
  - This is replicated to the followers, which is appended if there is majority

- Now similar to close, the edits made while using the function are committed only if a majority of the active nodes acknowledge. The temp file is copied, synced and then closed
- Version is updated
- To implement the prime finding functionality, the following setup is used
  - `FilterUniquePrimes` is used to pull the primes from the prime set
- For testing purposes, we use `TestAuth`, which is a way to quickly check the metadata

## client\_basics.go

- This code handles the client side logic
- Cache Management is implemented using `evictFromCache`, which allow the client to delete cache if the file is closed + there are no unsaved changes + not in use
- Functions seen in the server side, such as `read`, `write`, `close` and `open` are implemented here. The client also has the ability to `commit`
- The option for dirty writes, ie, writes that don't immediately go to the server, is also here
- There are many helper functions, such as
  - `extractLeaderAddr` is used by the client to connect to the new leader
  - `reconnectToLeader` is used to reconnect to the leader in case connection fails

## common.go

- This file handles a lot of misc. operations
- It sets up the file modes, such as read, write and readwrite
- It defines the chunk size, which is 4kB
- `santizePath` allows cleaner path traversal, which makes it easier for the functions to work
- `readLocalFile` is used to read a certain size of the file
- `writeLocalFile` is used to write data to the file at a certain offset
- Prime number checking is implemented using modular arithmetic
- `parseNumbers` is used to extract numbers from our input file

## connection.go

- This is the setup for the distributed file system.
- It implements the primary backup [Distributed File System](#)
- Using `NewServer`, it creates a new server instance, making sure to set it as the backup server
- Using `StartServer`, it registers the server in the main map and then proceeds to sync with the leader
  - Further functions are used to set up the background services and the directories
- `DialClient` is used to connect to the clients
  - The server loops over the given addresses, which are then attempted to connect
  - The server tries to find which node is the leader

## server\_replication.go

- This is the file used for server replication purposes

- Structs such as `heartbeat` and `LogEntry` are used
- It has the following functions
  - `SendHeartbeat`, which tracks servers, enforces leader priority and maintains the leader state
    - The system is locked during this process
    - Failure detection takes place, which checks two cases
      - One if the server is new, where it is added to the map
      - Other when the server is already known, where it is marked as alive
    - Leader validation takes place, giving the priority to the higher priority leader
    - Once the new leader is selected, the acceptance is sent.
      - Another heartbeat is sent, which is used for timeout/failure detection
  - `sendHeartbeat` is used by the peers
    - It follows a similar naming format to [Lab 1 Readme](#), where the server uses capitalised names, and the clients/peers use lowercase
    - It creates a stub to the clients, checking if the nodes are rejecting it or not
      - If even one node rejects, then it is no longer the leader
    - This and the above function combine to form a system with [Feedback](#), ensuring that only the node with the highest priority becomes the leader
  - `StartHeartbeat` keeps the leader alive in the system, by periodically starting heartbeats
  - `MonitorPrimary` is used as a trigger to check if the current leader ever stops sending heartbeats. If heartbeats are stopped, then an election is triggered
    - If the leader is dead, the system will mark it as dead
    - Once the election is over, a new node is marked as the leader
  - `OnHeartbeat` is used by the nodes to check who is the leader
  - `electPrimaryServer` is used by nodes to pick the leader who has the smallest ID
  - `appendToDisk` is used to append a log entry, ensuring durability if a crash occurs
  - `recoverFromLog` opens the log file, which is then processed and then rebuilt in case of a crash
  - `RebuildPrimeSet` creates an in-memory list of the prime numbers, but only for the primary node
  - `getOrCreateFileMeta` returns or creates the metadata of a file
  - `apply` executes a log entry by applying file operations, updating metadata and rebuilding derived state to keep everything consistent
  - `ApplyUpdate` is used to take updates from another node and then bring the server to sync
    - It unlocks before the heavy work, which ensures that the operation `RebuildPrimeSet` doesn't block other tasks - it is quite a heavy operation
    - It replaces everything with the leader's state, and updates the log entries
  - `AppendEntries` is the follower side logic, allowing us to implement a system similar to the Raft consensus algorithm
    - It runs on the follower, and it makes sure to append new log entries, which are then persisted and then committed
  - `applyCommitted` takes the committed log entries and then executes them on the state machine

- `sendAppendEntry` sends a single log entry to the followers and checks if the append operation was successfully executed
  - It sets up the `gRPC` connection and sets up the client stub
  - It sends the request of the log entries that should be appended
  - The function then verifies if the log entry was appended
- `HandleWrite` is the leader node's write path
  - It creates a log entry, which is then persisted locally
  - This is then replicated to peers
  - Once a majority of the peers acknowledge, the commit is applied
- `RequestRecovery` is used to set a follower to take a full state update
  - Essentially, a snapshot is taken and then sent to the follower so that it can be put back in sync
- `RecoverFromLeader` is run by a follower to put it back in sync
- `Start` and `Init` are used to start the communication
- `GetLeader` is a function used by the leader to let everyone know who the leader is

## Design Choices

- We have chosen a primary backup model
  - This means that only the leader has the ability to handle requests such as `create`, `write`, `open`, `delete`, `close`
  - Followers replicate the state and act as backup
  - Due to the robust backup system, even though there is just one leader, if the leader fails, the system will be able to easily recuperate and continue its normal functioning
- Replication is implemented using log files
  - The leader writes to a local log, which is later sent to followers. After a majority acknowledgement is reached, the log is committed
  - This is a simplified implementation of the Raft system
  - Write consistency is ensured by the majority voting
- Fault tolerance is implemented in many ways, such as
  - Incomplete writes being deleted after a crash
    - Furthermore, entry and lock conditions are reset, preventing deadlock
  - Locks are released outside the mutex.
    - If these locks are released inside the mutex, deadlocks and blocks could occur
  - Heavy usage of `s.mu` and `fm.mu`, which lock the system at important points, such as getting access to files
  - Heartbeats are used to ensure liveness
    - The servers maintain a map of the active nodes
    - If a heartbeat is missing, the node is marked as dead
    - Since the size of the system is small, such a solution works quite effectively

- Concurrency is controlled via locks and leases. Files' access can only be given if the access read/write and can read/write tokens are passed to a particular node
  - Using locks ensures correctness since only one node can have the access at a time
  - Having the lease system allows the server to clean up nodes that crashed while accessing a particular file
- New server initializations are put in backup position, which is so that there aren't multiple leaders
- Idempotency is implemented using checking the cache of requests
- The prime set is rebuilt in case of crashes, which allows the logic to be cleaner as well as storing the data
  - However, the re-computation is quite intense
- Guaranteeing atomicity in a distributed system is quite hard. Our approach was to have the node write to a temporary file, which is then copied to the actual file once the function is complete. This allows us to have both atomicity and crash safety
- Every update is logged using the `appendToDisk` function, which is then replayed on restart using the `recoverFromLog`
  - This allows the system to recover in case of crashes, as it can reconstruct the state by reading the system
  - This is how we implemented Write Ahead Logging
- File reading is done via 4KB chunks, which is both memory efficient and scalable for the prime finding application
- The client maintains cache using dirty writes and cache eviction, which improves performance
  - This means that the client keeps a local copy, which allows the client to access
  - Dirty writes allow for better performance, as once the chunk is no longer in use, all the edits can quickly be combined and then written into the file
  - Cache eviction ensures that the client side memory is controlled and that repeat requests don't end up slowing down the system
- Periodically, using the `cleanupRequests` and `cleanupLeases` functions, the system cleans up failed nodes, memory leaks and just generally allowing the system to function better
- To implement snapshots, we are taking advantage of the fact that our system locks itself for implementing function such as `cleanupLeases`. The leader node will send a message to all clients, locking the system and asking to record the state and send it to the leader, which will then be recorded and presented as a snapshot.